Operating Systems

File System

Filesystems

- File systems provide efficient and convenient access to the disk by allowing data to be stored, located, and retrieved easily
- A file system poses two quite different design problems
  - Defining how the file system should look to the user. This task involves defining a file and its attributes, the operations allowed on a file, and the directory structure for organizing files
  - Creating algorithms and data structures to map the logical file system onto the physical secondary-storage devices
- File system manages metadata information. Metadata includes all of the file-system structure except the actual data
- A file control block (FCB) (an inode in UNIX file systems) contains information about the file, including ownership, permissions, and location of the file contents

Files

- A file is a named collection of related information that is recorded on secondary storage.
- It is basically a logical storage unit.
- Files are mapped by the operating system onto physical devices
- There are several types of files which is generally indicated by extension of file name
- **File Attributes:**
    - Name
    - Identifier
    - Type
    - Location
    - Size
    - Protection
    - Time, date, and user identification

File Operations

- Creating
- Opening
- Writing
- Reading
- Repositioning within a file
- Deleting
- Truncating

Information Associated with Open Files

- File Pointer
- File Open Count
- Location of the file
- Access Rights

Access Methods

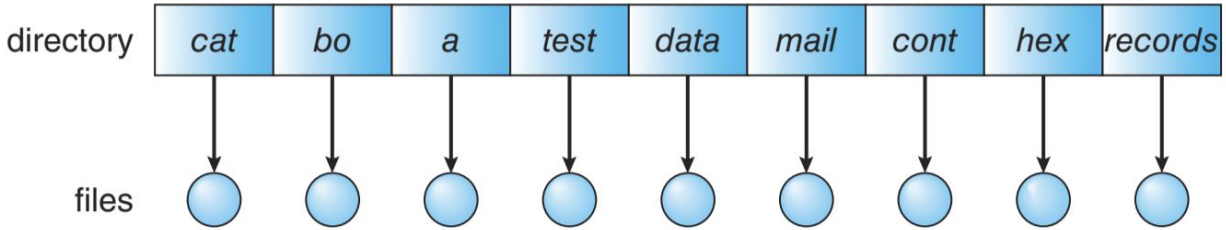
- Sequential Access: Information in the file is processed in order, one record after the other
- Direct/Relative Access: a file is made up of fixed-length logical records that allow programs to read and write records rapidly in no particular order.
- Other Access Methods: can be built on top of a direct-access method. Ex: Index-based Access

Directories

- The directory can be viewed as a symbol table that translates file names into their directory entries
- Operations on a directory
  - Search for a file
  - Create a file
  - Delete a file
  - List files
  - Rename a file
  - Traverse the filesystem
- A directory itself can be organized in many ways
- The organization must allow us to insert entries, to delete entries, to search for a named entry, and to list all the entries in the directory
- Tree-Structured Directories are commonly used in most Operating Systems

Directory Structure

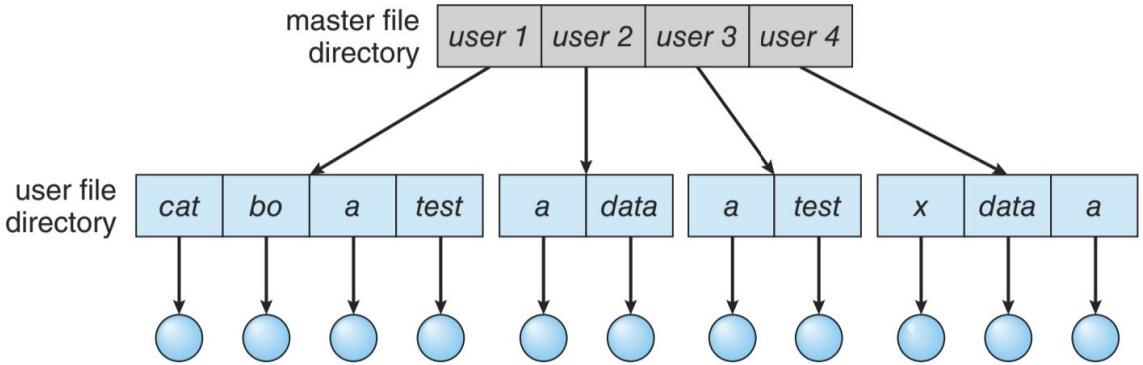
- **Single Level Directory**



Directory Structure

Two Level Directory

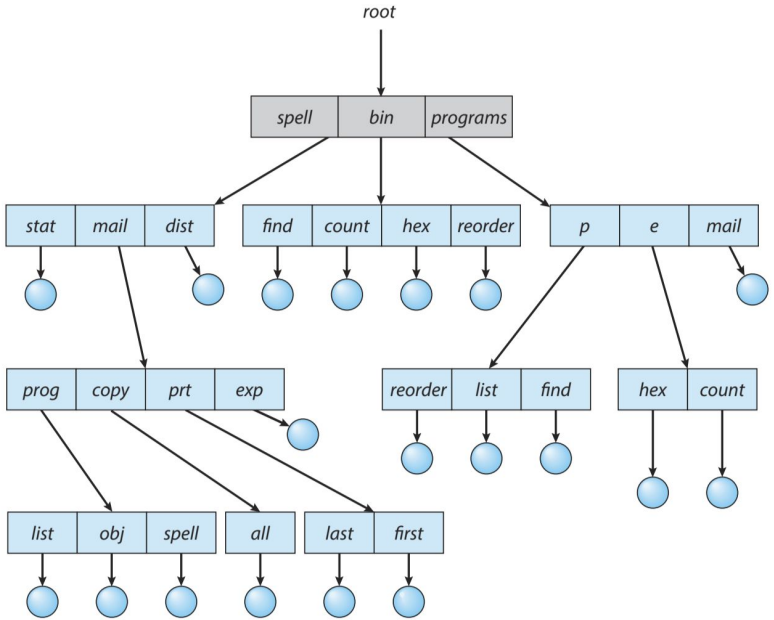
- Solves name collision problem
- Isolates one user from another



Directory Structure

Tree-Structured Directory

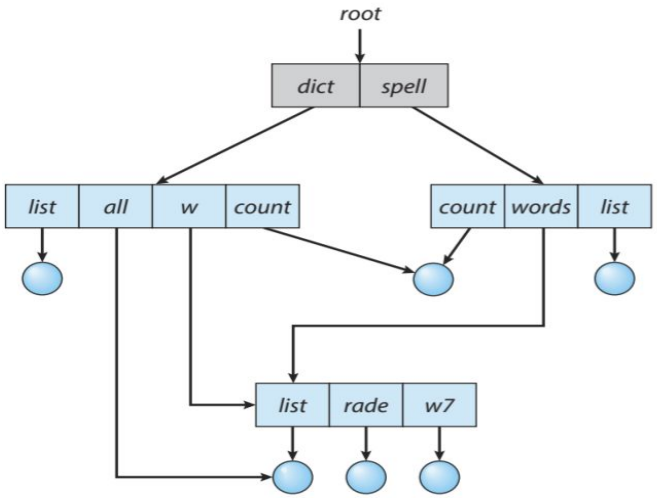
- Allows users to create their own subdirectories and to organize their files accordingly
- Normally, each process has a **current directory**
- When reference is made to a file, the current directory is searched
- Path names can be of two types:
  - absolute path name: Starts at the root directory
  - relative path name: starts from the current directory
- Policy options on deletion of a directory:
  - Allow deletion only if the directory is empty
  - To delete a directory, delete all its files and subdirectories



Directory Structure

Acyclic-graph Directory

- Allows directories to share subdirectories and files
- A shared file (or directory) is not the same as two copies of the file. Instead, only one actual file exists, so any changes made by one person are immediately visible to the other
- One common way to implement shared files is to use **links**
- A **link** is effectively a pointer to another file or subdirectory
- If the directory entry is marked as a link, then the name of the real file is included in the link information
- Another common approach to implementing shared files is simply to duplicate all information about them in both sharing directories
- A major problem with duplicate directory entries is maintaining consistency when a file is modified
- An acyclic-graph directory structure is **more flexible** than a simple tree structure, but it is also **more complex**



Directory Structure

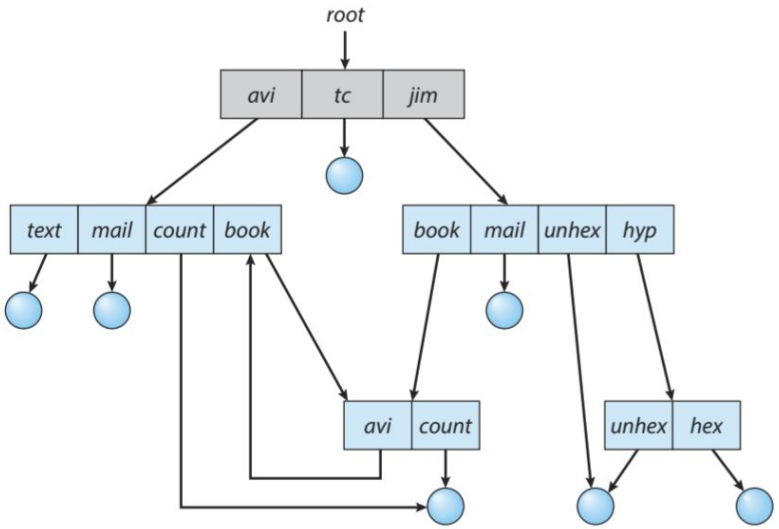
Acyclic-graph Directory

- A file may now have multiple absolute path names. This is problematic while traversing the filesystem
- Another problem involves deletion
- One possibility is to remove the file whenever anyone deletes it, but this action may leave dangling pointers to the now-nonexistent file
- In a link-based implementation, deletion of a link need not affect the original file; only the link is removed
- If the file entry itself is deleted, it will result in dangling pointers
- In that case we can leave the links until an attempt is made to use them
- the system designer should consider carefully what to do when a file is deleted and another file of the same name is created, before a symbolic link to the original file is used
- Another approach to deletion is to preserve the file until all references to it are deleted

Directory Structure

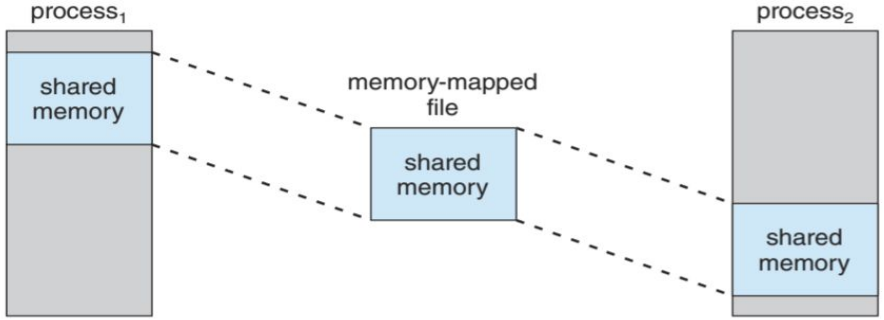
General Graph Directory

- If cycles are allowed to exist in the directory, we want to avoid searching any component twice, for reasons of correctness as well as performance
- Deletion of file also poses a challenge as there is a possibility of **self-referencing**
- In this case, we generally need to use a **garbage collection**



Protection

- Protection mechanisms provide controlled access by limiting the types of file access that can be made.
- Access is permitted or denied depending on several factors, one of which is the type of access requested
- Some examples of the operations that can be controlled are:
  - Read, Write, Execute, Append, Delete, List, Attribute Change
- The most common approach to the protection problem is to make access dependent on the identity of the user.
- The most general scheme to implement identity dependent access is to associate with each file and directory an **access-control list (ACL)** specifying user names and the types of access allowed for each user
- This approach has the advantage of enabling complex access methodologies.
- The main problem with access lists is their length. If we want to allow everyone to read a file, we must list all users with read access.
- This technique has two undesirable consequences
  - Constructing such a list may be a tedious and unrewarding task, especially if we do not know in advance the list of users in the system.
  - The directory entry, previously of fixed size, now must be of variable size, resulting in more complicated space management.

Memory-Mapped Files

- Memory mapping of a file allows a part of the virtual address space to be logically associated with the file
- It is accomplished by mapping a disk block to a page (or pages) in memory
- In Linux it is done by using the mmap() system call
- writes to the file mapped in memory are not necessarily immediate (synchronous) writes to the file on secondary storage.
- Generally, systems update the file based on changes to the memory image only when the file is closed
- Multiple processes may be allowed to map the same file concurrently, to allow sharing of data.
- Writes by any of the processes modify the data in virtual memory and can be seen by all others that map the same section of the file.